Praktikum Mobile Systeme und Cloud · winter semester 2026/27 · Hochschule Albstadt-Sigmaringen

Sensor Lab

Task 1 – Sensors

Name and matriculation number: on the submitted copy

2026-11-01

Code and full documentation:
https://github.com/dpfx/mobile-systems-and-cloud/tree/aufgabe-1-submission/aufgabe-1

Development environment

Device	No physical device: Android Emulator, AVD “Medium Phone API 37.0” (system image `sdk_gphone64_x86_64`)
Android version	Android 17 (API 37)
AI tool and model	Claude Code 2.1.309–2.1.310 (VS Code extension) with the model Claude Opus 5.5; the app was generated by a Claude Code sub-agent with the same model

Host system and build toolchain: `docs/environment.md`.

Specification – Sensor Lab

Hiking companion: measures movement, altitude change and distance walked, warns of strong shocks; every value must be verifiable. Android, Kotlin, one screen, minSdk 26, compile/targetSdk 37.

1. Sensors, quantities, rates

Sensor	Displayed (units)	Requested rate
Accelerometer	x, y, z and |a| = √(x² + y² + z²) in m/s²; |a| also in g (1 g = 9.80665 m/s²)	`SENSOR_DELAY_GAME` (20 ms); switch: `SENSOR_DELAY_NORMAL`
Barometer	pressure (hPa); altitude change Δh since start (m)	`SENSOR_DELAY_NORMAL` (200 ms)
GPS (`LocationManager`)	latitude, longitude (°, 5 decimals); accuracy (m); distance since start (m)	1 fix per second

- Δh: `SensorManager.getAltitude()` (standard atmosphere), relative to the start.
- Distance uses only fixes with accuracy ≤ 20 m and grows only when the step from the last counted fix exceeds the new fix's accuracy.
- **Reset** restarts Δh and distance. Each sensor shows its requested and measured event rate (Hz over the last 2 s, from event timestamps).

2. Display modes

Tabs **Values** (card per sensor: values, requested/measured rate, status) and **Chart** (Vico `compose-m3:3.3.1`: one line chart each for |a|, Δh and distance over the last 60 s). History: 60 s ring buffer, ≤ 10 points/s; UI updates ≤ 10 times/s, independent of sensor rates.

3. Warning rule

|a| ≥ 25 m/s² (≈ 2.5 g) triggers an in-app banner "Strong shock detected" (time, peak |a|) and, if the **Shock alerts** switch is on, a notification; at most once per 30 s. Turning the switch on (off by default) requests `POST_NOTIFICATIONS`. At `SENSOR_DELAY_NORMAL` the card notes reduced reliability.

4. Missing sensors and permissions

- Missing sensor: "Not available on this device" on its card, chart hidden, the rest keeps working.
- Location (fine + coarse) via **Enable GPS** on the GPS card. Denied: explanation + **Grant** (opens app settings after permanent denial). Approximate only: no distance. GPS off: the card says so. Permissions are re-checked whenever the app returns to the screen.
- Notifications denied: banner only, hint at the switch. Never block the screen or ask repeatedly.

5. Background

Sensors and GPS run only while visible: they stop 5 s after leaving the screen (survives rotation) and resume on return. Tracking pauses, also with the screen off (accepted); the chart shows a gap, distance resumes from the next fix. No background location, foreground service or disk storage.

6. Logging

Logcat, debug builds only: `SensorLab.Sensor` (raw events, measured rate), `SensorLab.Process` (derived values, fix decisions, warnings), `SensorLab.UI` (displayed values). At most one line per sensor and tag per second; warnings and state changes always. Positions rounded to 3 decimals.

7. Architecture

Repository per source (`callbackFlow`, `awaitClose` unregisters); `onSensorChanged()` only forwards. ViewModel: `StateFlow` via `stateIn(WhileSubscribed(5000))`, no `Context`, constructor injection. UI: `collectAsStateWithLifecycle()`. Only AndroidX, Compose (BOM, Material 3), Vico, all pinned.

Spec amendments

The specification above is the prompt the app was generated from, so it stays unchanged. Changes found necessary later are amendments; where they differ, the amendment applies.

#	Section	Change	Reason	Finding
A1	7 Architecture	`kotlinx-coroutines` added to the allowed libraries	The section requires `callbackFlow` and `StateFlow`, which come from this library, but its library list left it out	F2
A2	3 Warning rule	Tapping the notification opens the app	Android: “Every notification must respond to a tap” (Build a notification)	F4
A3	1 Sensors	A GPS step counts only if it also implies at most 10 m/s; a faster step adds nothing and its fix becomes the new reference	Without a plausibility limit, a position jump was counted as 9,359,974 m	F1

Full wording (original and amended): `docs/spec-amendments.md`.

Part C – Code review

Reviewed version: the raw AI output, commit `ea661d8`; code locations refer to it, relative to `app/src/main/java/de/penguin/sensorlab/`. **Method:** each checklist item was checked in the code against our coding guidelines and, where reading isn’t enough, on the emulator (Android 17) with the app’s log, Android’s event log and `dumpsys`.

#	Checklist item	Verdict	Findings
1	Lifecycle: listeners unregistered when not visible	correct	–
2	Permissions: requested at runtime, denial handled	correct	–
3	Missing sensors: no crash	correct	–
4	Units and formulas	correct, but distance without plausibility check	F1
5	Sampling rate and energy	appropriate	–
6	UI and memory: redraw per reading, unbounded lists	correct	–
7	APIs and libraries: deprecated or non-existent	correct, but one library not declared	F2
8	Privacy: location stored or logged	correct	–

Beyond the checklist: findings F3–F5 (section 9). All five findings were fixed, one commit each.

1. Lifecycle – correct

- **Code:** the UI collects the state only while the screen is visible (`collectAsStateWithLifecycle()`, `ui/SensorLabScreen.kt:58`); all sources run behind `stateIn(…, WhileSubscribed(5000))` (`ui/SensorLabViewModel.kt:85-97`), and cancelling them runs `awaitClose`, which unregisters the listeners (`data/SensorRepository.kt:58-61`, `data/LocationRepository.kt:105-108`).
- **Emulator:** Home at 13:55:00.474 → `13:55:05.479 … Accelerometer: listener unregistered`, barometer and GPS within 5 ms; back in the app, all three are registered again; a rotation recreates the screen without unregistering anything.

2. Permissions – correct

- **Code:** location is requested only via **Enable GPS**, notifications only when **Shock alerts** is turned on (`ui/ValuesTab.kt:136-142`, `ui/SensorLabScreen.kt:101-110`); after a permanent denial, **Grant** opens the app settings (`ui/SensorLabScreen.kt:112-125`); permissions are re-checked on every return to the screen (`ui/SensorLabScreen.kt:64-66`).

- **Emulator:** a denial shows an explanation and **Grant**; after a permanent denial, **Grant** opens App info; approximate location shows the position but no distance; with notifications blocked, the warning appears as a banner only, with a hint at the switch; revoking location in the background causes no crash.

3. Missing sensors – correct

- **Code:** `getDefaultSensor()` may return `null`; then nothing is registered and the card shows "Not available on this device" (`data/SensorRepository.kt:30-33`, `ui/ValuesTab.kt:54-55`).
- **Emulator** with the barometer removed (`hw.sensors.pressure=no`): barometer card "Not available on this device", Δh chart hidden, the accelerometer at 50 Hz and the shock warning still working, 0 crashes.

4. Units and formulas – correct, one finding

Quantity	Code	Formula / unit
x, y, z	`ui/SensorLabViewModel.kt: 190-192`	`SensorEvent.values` in m/s²
|a|, in g	`ui/SensorLabViewModel.kt:193`, `ui/Format.kt:21`	√(x² + y² + z²); divided by `SensorManager.STANDARD_GRAVITY` (9.80665)
Pressure	`ui/SensorLabViewModel.kt:265`, `ui/ValuesTab.kt:111`	`SensorEvent.values[0]`, which Android defines as "Atmospheric pressure in hPa (millibar)" (`SensorEvent`); shown in hPa
Δh	`ui/SensorLabViewModel.kt: 269-277, 376-377`	`getAltitude(PRESSURE_STANDARD_ATMOSPHERE, p)` minus the value at the start
Distance	`processing/DistanceTracker.kt: 79-83`	`Location.distanceBetween` (WGS84)
Measured rate	`processing/RateMeter.kt:21-26`	(events − 1) / time span within 2 s, from the events' timestamps

Repeated measurements with known inputs: Part D.

F1 – GPS distance has no plausibility check

Location	`processing/DistanceTracker.kt:60-65`
Problem	Every step longer than the fix's accuracy is added, however implausible. The code follows the spec, which sets no limit: a spec gap, not an AI error.
Impact	On the emulator, a jump from the default position (Mountain View) to the test position within 1.7 s was counted as **9,359,974 m** (`counted: step 9359974.0 m > accuracy 5.0 m`).
Fix	A step that implies more than 10 m/s is not counted, and its fix becomes the new reference; spec amendment A3. Verified: a jump to Mountain View now gives `rejected: … = 1336250.1 m/s > 10 m/s, new reference`, and the distance stays at 111.2 m. Commit `6638a12`.

5. Sampling rate and energy – appropriate

The accelerometer runs at `GAME` (50 Hz) because the shock warning needs it: at 5 Hz the samples are 200 ms apart, so a short peak can fall between two of them. The barometer runs at `NORMAL`, the slowest preset, since pressure changes slowly; GPS delivers 1 fix per second and runs only while the screen is visible. The requested periods reach Android unchanged (`dumpsys sensorservice`: `samplingPeriod= 20000us` / `200000us`). At `NORMAL`, Android delivers 15 Hz instead of 5 Hz (Part D, section 4).

6. UI and memory – correct

- **Code:** the screen gets a new state snapshot every 100 ms, not per reading (`ui/SensorLabViewModel.kt:85-92`); the chart history is a ring buffer of 600 points per quantity (`processing/RingBuffer.kt`, `processing/History.kt:21`).
- **Emulator:** 2 frames rendered in 10 s on the idle Values tab and 99 on the Chart tab (≈ 10/s, per snapshot, not per reading); the Java heap stayed between 12.0 and 16.5 MB over 2 minutes, with no upward trend.

7. APIs and libraries – correct, one finding

The project builds cleanly from scratch, a full recompile shows 0 compiler warnings (so no deprecated APIs), and all pinned versions were current on 2026-10-25 (AGP 9.4.1, Kotlin 2.4.21, Compose BOM 2026.09.00, Vico 3.3.1).

F2 – `kotlinx-coroutines` is used but not declared or pinned

Location	`app/build.gradle.kts`, `gradle/libs.versions.toml`; used in 16 imports, e.g. `data/SensorRepository.kt:8-11`
Problem	The library reaches the app only indirectly through AndroidX, so its version is whatever Gradle resolves (1.11.0); spec section 7 requires every library to be pinned. Root cause is a spec gap: the spec requires `callbackFlow` and `StateFlow`, which come from this library, but its library list leaves it out, and the AI resolved this silently.
Impact	No effect today; any AndroidX update can change the coroutines version unnoticed.
Fix	Declared with version 1.11.0; spec amendment A1. Verified: Gradle lists it as a direct dependency at 1.11.0, and the build succeeds. Commit `ef533ad`.

8. Privacy – correct

- **Code:** no storage API anywhere; positions are logged with 3 decimals only (`ui/SensorLabViewModel.kt:302-306`, `ui/ValuesTab.kt:194`), and logging is off in release builds (`log/ChainLog.kt:21`).
- **Emulator:** the app's data folder holds only an AndroidX marker file; 0 of 1,370 log lines with a position have more than 3 decimals.

9. Beyond the checklist

F3 – A missing sensor is not logged

Location	`ui/SensorLabViewModel.kt:153, 163, 170`
Problem	If a sensor or GPS is missing, its source returns without a log line, although spec section 6 requires state changes to be logged always.
Impact	With the barometer removed, the log says nothing about it, so it can't tell a missing sensor from one that sends no events.
Fix	Each source logs "not available on this device". Verified: `Barometer: not available on this device` with the barometer removed. Commit `249f929`.

F4 – Tapping the notification opens a second screen with fresh data

Location	`data/ShockNotifier.kt:50-55`
Problem	The notification starts `MainActivity` without launch flags, so Android creates a second instance with its own ViewModel. Opening the app from the notification isn't in the spec; the AI added it.
Impact	With the app open, a tap created a new instance on top (`wm_on_create_called`). By the code, it starts with Δh and distance at 0, Shock alerts off and an empty chart, while the real data sits underneath.
Fix	The notification uses the app's launcher intent, which brings the existing screen to the front; spec amendment A2. Verified: a tap leaves the same instance in front, with no new `wm_on_create_called`. Commit `ad46acf`.

F5 – No app icon

Location	`app/src/main/AndroidManifest.xml:21` (no `android:icon`)
Problem	The project skeleton, created by the AI, has no launcher icon, although it was described as equivalent to Android Studio's template. Lint: `MissingApplicationIcon`.
Impact	Android's generic placeholder icon on the home screen and in the notification.
Fix	An adaptive vector icon. Verified: Lint no longer reports `MissingApplicationIcon`. Commit `0e2987c`.

Full version with all excerpts and emulator tables: `docs/review.md`.

Part D – Verification by measurement

Method: no physical device was available, so every experiment injects known values into the emulator (`emu sensor set`, `emu geo fix`); the script `tools/measure.sh` then reads what the app received, calculated and displayed, 5 runs per scenario (raw data). This verifies the app's processing chain (sensor event → processing → display) against a known input, not the accuracy of real sensors, so a deviation comes from the app or from the emulator. Tested version: commit `0e2987c` (after fixes F1–F5); times in excerpts are emulator time (UTC).

1. Accelerometer

Setup: from the phone upright at rest, inject the scenario's acceleration and read the app's |a| from its log 2.5 s later.

Expected: flat on a table, the `SensorEvent` reference gives "+9.81" (gravity, so |a| = 1 g); tilted, gravity splits into x = g · sin 30° and z = g · cos 30°, and |a| = g · √(sin² 30° + cos² 30°) stays 1 g; in free fall the accelerometer "reads a magnitude of 0 m/s^2" (same reference).

Scenario	Injected (x, y, z) in m/s²	Expected |a|	App: |a| (all 5 runs)	Deviation
Flat on a table	(0, 0, 9.80665)	9.807 m/s² = 1.000 g	9.807 m/s² = 1.000 g	0
Right edge raised by 30°	(4.903325, 0, 8.492808)	9.807 m/s² = 1.000 g	9.807 m/s² = 1.000 g	0
Free fall	(0, 0, 0)	0	0.000 m/s² = 0.000 g	0

Explanation: the emulator passes the values on unchanged, so this confirms the axes, the magnitude formula and the conversion to g (9.80665 m/s²), not the noise or offset of a real sensor.

2. Barometer

Setup: tap **Reset** at 1013.25 hPa, then inject the pressure of a position 3, 6 or 9 m higher (1–3 floors), computed by inverting Android's altitude formula h = 44330 · (1 − (p / p₀)^(1/5.255)) (source).

Expected: Δh = 3.00 / 6.00 / 9.00 m, by construction; the app computes Δh with that formula (spec section 1).

Floors	Injected (hPa)	Received (hPa)	App: Δh (all 5 runs)	Formula for the received pressure	Expected	Deviation
1	1012.890	1012.875	3.12 m	3.12 m	3.00 m	+0.12 m
2	1012.530	1012.500	6.25 m	6.25 m	6.00 m	+0.25 m
3	1012.169	1012.125	9.37 m	9.37 m	9.00 m	+0.37 m

Explanation: the app is exact for the pressure it receives. A pre-test with 21 pressures in steps of 0.01 hPa showed that the emulator rounds pressure to the nearest multiple of 0.125 hPa (`barometer-steps.csv`). One floor of 3 m is 0.360 hPa; the emulator delivers 3 steps = 0.375 hPa, so every floor appears 4.1 % too high.

3. GPS

Setup: tap **Reset** at 48.2120° N, 9.0270° E; walk 10 steps of 0.0001° north, one every 4 s (2.8 m/s); then 20 s of jitter 2.2–3.3 m around the end point; then a jump to Mountain View (more than 9,000 km). The emulator reports an accuracy of 5.0 m for every fix.

Expected: the walk is 111.19 m: `Location.distanceBetween` uses "the WGS84 ellipsoid", and 0.001° along the meridian at φ = 48.21° is M · 0.001 · π / 180 with M = a (1 − e²) / (1 − e² sin² φ)^1.5 = 6,370,973 m. Jitter fixes stay within their accuracy, so no step counts (spec section 1); the jump implies far more than 10 m/s, so it is rejected (amendment A3).

Phase	Expected	Measured (all 5 runs)	Deviation
Walk	111.19 m	111.2 m	none beyond the display's 0.1 m
Jitter	0 steps counted	0 steps counted, 111.2 m	0
Jump	rejected, distance unchanged	111.2 m	0

Explanation: distance, accuracy filter and plausibility limit work as specified. Single steps alternate between 10.9 and 11.3 m in the emulator (one extra walk, `gps-steps.log`; mechanism not determined), but after every second step the distance is on target. A first series with a script error (first step too fast, correctly rejected by the 10 m/s limit) was repeated (`ai-errors.md`, entry 15).

4. Event rate: `SENSOR_DELAY_GAME` vs. `SENSOR_DELAY_NORMAL`

Setup: accelerometer at `GAME`, then at `NORMAL` (**Low rate** switch); 5 readings each of the rate the app shows (events in the last 2 s) and of Android's own record of the last 50 events (`dumpsys sensorservice`).

Requested: `GAME` = 20,000 µs = 50 Hz, `NORMAL` = 200,000 µs = 5 Hz (Sensors Overview).

Setting	Requested	App shows (runs 1–5)	Android's record (runs 1–5)	Measured	Deviation
`GAME`	50 Hz	50.0 Hz in all runs	50.0 Hz in all runs	50.0 Hz	0
`NORMAL`	5 Hz	15.0 / 15.0 / 15.0 / 24.2 / 15.0 Hz	15.0 / 15.0 / 15.0 / 23.9 / 15.0 Hz	mean of 5: 16.8 Hz; without run 4: 15.0 Hz	+10.0 Hz (3 ×)

Explanation: "The delay that you specify is only a suggested delay. The Android system and other applications can alter this delay" (Sensors Overview). Android runs the sensor at the fastest period any client requests (`sensorservice.txt`, taken at `GAME`):

```
0x00000000) active-count = 3; sampling_period(ms) = {66.7, 200.0, 20.0}, selected = 20.00 ms; …
14:54:16 + … samplingPeriod=   66667us … (…, com.android.server.wm.WindowOrientationListener$AccelSensorJudge)
15:33:15 + … samplingPeriod=   20000us … (…,
com.google.android.location.collectionlib.SensorScanner$SensorEventListenerAdapter)
```

The system's screen-rotation detection requests 66,667 µs (`SENSOR_DELAY_UI`, Android source), so at `NORMAL` the sensor runs at 15.0 Hz, exactly what both values show. Run 4: for less than a second at 15:33:15, a Google Play services component requested 20 ms; it is the only disturbance during the whole `NORMAL` series, and run 4 is the only reading that deviates.

5. Consequence for energy use

Setup: CPU time of the app process (from `/proc/<pid>/stat`) over 60 s, alternating `GAME` and `NORMAL`, 3 runs each; 100 % means one CPU core busy all the time.

Setting	Events/s received	CPU time, runs 1–3	Mean
`GAME`	50	3.27 / 3.12 / 3.21 %	3.20 %
`NORMAL`	15	2.68 / 2.73 / 2.61 %	2.67 %

70 % fewer events cost only 16.5 % less CPU time: 83 % of it remained.

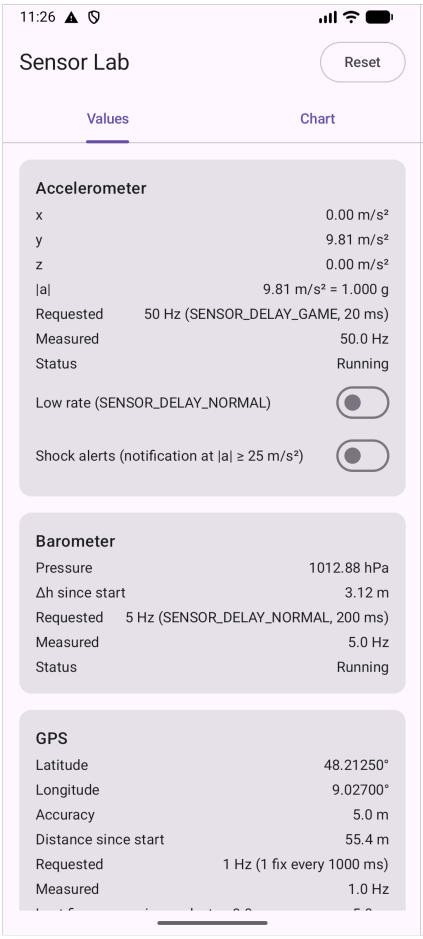
What follows for energy use:

1. **The requested rate limits the savings, it doesn't guarantee them:** `NORMAL` delivered 15 Hz instead of 5 Hz because a system service asks for more; and at `GAME`, Sensor Lab makes the sensor run at 50 Hz for every client.
2. **For this app, the rate is a small lever.** The larger one is not running at all when nothing is visible, as the app does (Part C, item 1), and as the `SensorManager` reference advises: "Always make sure to disable sensors you don't need, especially when your activity is paused." With the screen off, batching would be the lever (`maxReportLatencyUs`: it reduces "the number of interrupts the AP (Application Processor) receives, hence reducing power consumption"); the emulator's sensor can't batch (`sensorservice.txt`, line 8), so this wasn't measured.
3. **`GAME` stays right for the shock warning:** `NORMAL` would save little and could miss short peaks.

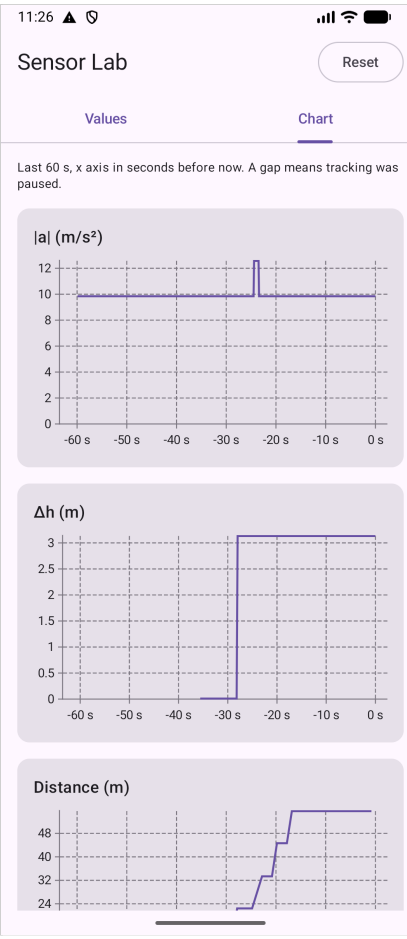
Limit: the emulator runs on a PC processor, so only the ratio carries over, not the absolute values. Full version with all excerpts: `docs/measurements.md`.

Screenshots

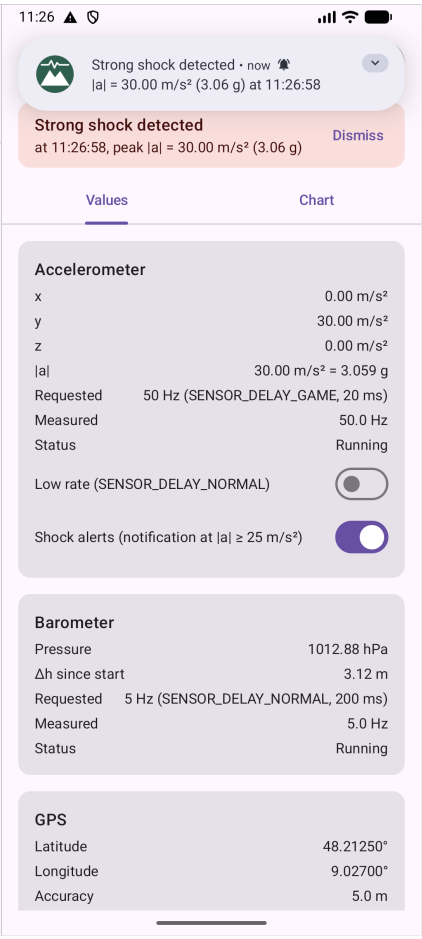
Taken on the emulator (see the development environment) with injected values: after **Reset**, the phone at rest with one bump of about 12.5 m/s², the pressure of one floor higher and a walk of 5 GPS steps north; then one shock of 30 m/s² with **Shock alerts** on.



Display mode 1: Values tab



Display mode 2: Chart tab



Triggered warning: in-app banner and notification (|a| ≥ 25 m/s²)

Part E – Sensor data and AI

Scenario: the app is to recognise sitting, walking or climbing stairs from the accelerometer (read at 50 Hz, as now) and the barometer (one floor ≈ 0.36 hPa, Part D 2). The public UCI HAR dataset uses the same kind of data at the same rate (smartphone accelerometer, plus gyroscope, at 50 Hz) for these activities, in windows of 2.56 s.

1. Edge or cloud?

Recommendation: run the recognition on the device. The cloud is only used to train the model and to distribute model updates; users' raw sensor data doesn't leave the phone.

Criterion	On the device	In the cloud	Evidence
Latency	Result as soon as a window of data is complete and classified; no network involved	Same window, plus upload, processing and response	Window length: UCI HAR
Energy	Sensor data stays in memory; the work is computation only. For comparison, the whole app at 50 Hz uses 3.20 % of one CPU core (emulator)	The raw values alone are 600 B/s (3 axes × 4-byte `float` × 50 Hz), 2.2 MB per hour, sent continuously: "When fully active, the wireless radio consumes significant power, but when inactive or in standby, the radio consumes very little power"	Part D 5; `SensorEvent.values`; radio state machine
Offline	Works wherever the phone is	Recognises nothing without a connection; the app is a hiking companion, used outdoors, where coverage can't be assumed	Spec, first line
Privacy	Raw data stays on the phone; at most the result leaves it, if the user wants that	The raw accelerometer and barometer stream goes to a server. Accelerometer data alone can reveal health, body features and identity (section 2), while the recognition needs only the result	Kröger et al.

The sensors run in both variants, so the difference is computing on the phone vs. transmitting continuously. Latency decides least: activities last seconds to minutes, and both variants need a full window first. Training needs labelled data from many people who agreed to it, which fits the cloud; it doesn't need the users' sensor data. Android's Activity Recognition Transition API knows only "IN_VEHICLE ON_BICYCLE RUNNING STILL WALKING", so stairs and sitting need an app-specific model.

2. Personal data

Yardstick: GDPR Art. 4(1): "'personal data' means any information relating to an identified or identifiable natural person …; an identifiable natural person is one who can be identified, directly or indirectly, in particular by reference to an identifier such as a name, an identification number, location data …" (GDPR).

Data in Sensor Lab	Personal?	What it reveals about the person	Evidence
GPS position (shown with 5 decimals ≈ 1 m)	Yes, directly: "location data"	Where the person is; over time home, workplace, routes, routines	GDPR Art. 4(1)
Distance since Reset	Yes, derived from the positions	How far and how fast the person moves	Spec section 1
Positions in the debug log (3 decimals ≈ 100 m; debug builds only)	Yes, still location data	Where the person was and when, to about 100 m	Part C, item 8
Accelerometer x, y, z and |a|	Yes, it allows conclusions	"a device holder's location, activities, health condition, body features, gender, age, personality traits, and emotional state"; it can "uniquely identify a person based on biometric movement patterns"	Kröger et al., abstract
Shock warnings (time, peak |a|)	Yes	When the phone was hit or dropped (with GPS: where)	Spec section 3

Data in Sensor Lab	Personal?	What it reveals about the person	Evidence
Barometer pressure and Δh	Yes, mainly in combination	Floor changes, so the floor within a building; with GPS, which floor of which building	Part D 2
Activity labels of the planned feature	Yes	Daily routine, how much the person sits or moves; possibly the health condition	Kröger et al.

Not personal on their own: the measured event rates; they depend on the phone and on other apps (Part D 4).

Conclusion: every sensor in the app produces personal data, GPS directly, the accelerometer and barometer through what can be inferred from them, most strongly in combination. Android's permissions don't reflect this: Sensor Lab needs one only for location (and notifications), while the accelerometer and barometer can be read without asking; Android only limits the rate of motion sensors "to protect potentially sensitive information about users" (Sensor rate-limiting). Sensor Lab keeps all values in memory and stores nothing (spec section 5); the planned recognition fits this only if it runs on the device.

Sources: GDPR, Art. 4(1) · J. L. Kröger, P. Raschke, T. R. Bhuiyan: "Privacy Implications of Accelerometer Data: A Review of Possible Inferences", ICCSP '19, 2019, doi: 10.1145/3309074.3309076 · UCI HAR dataset · Android Developers: Activity Recognition Transition API, radio state machine, `SensorEvent.values`, Sensor rate-limiting. Full version: `docs/assessment.md`.

AI log – Task 1

Tool: Claude Code (VS Code, WSL) with the model Claude Opus 5.5, used throughout; the app itself was generated by a Claude Code sub-agent with the same model. **Inputs the AI worked with:** the Part A spec plus a short process wrapper as the generation prompt (`generation-prompt.md`); the `CLAUDE.md` files (course rules, working rules), loaded automatically in every session; `guidelines.md` (pinned best practices with sources, also the review checklist).

Tool, model	What for?	Key prompts (summary)	Adopted, changed or discarded?	AI errors and own corrections
Claude Code, Opus 5.5	Part A: task, spec, working rules, guidelines	Explain the task PDF; Kotlin best practices; “what do we want to achieve with the app?”; one decision at a time; spec down to one page	Spec text drafted by the AI, every decision made by the student; guidelines adopted	Unrealistic fall scenario as the reason for the notification → challenged, warning renamed to “Strong shock detected” (5); five decisions at once → rule: one at a time (3)
Claude Code, Opus 5.5	Part B: project skeleton	Git before the skeleton; app ID without the student’s name	Adopted; API level changed	Compiled against API 36, build failed → API 37 (2); no app icon → added (13, F5)
Claude Code sub-agent, Opus 5.5	Part B: generate the app	Spec as the prompt, plus: work alone, build until it succeeds, list every assumption; guidelines as coding rules	Adopted unchanged as a raw commit, then five review fixes, one commit each	Notification tap opened a second screen with fresh data → launcher intent (F4); missing sensor not logged → log line (F3)
Claude Code, Opus 5.5	Part C: review support, emulator checks	One checklist item at a time; extend the frozen spec with amendments instead of editing it	Verdicts and findings checked by the student	Test commands stacked copies of the app’s screen and ran steps from the wrong state → launch like the launcher, clean state per step (11, 12)
Claude Code, Opus 5.5	Part D: measurement script, evaluation	Inject ideal values, 5 runs each; keep the GPS step sizes in the report; read the whole PDF to keep the scope right	Script and report adopted after fixes	First GPS step too fast, 2 of 5 runs wrong → script fixed, series repeated (15); wrong explanation of the GPS step sizes → dropped (16)
Claude Code, Opus 5.5	Part E, documentation	“what does the task description say?”; decide edge vs. cloud	Recommendation decided by the student (device; cloud only for training)	Proposed research the task doesn’t ask for → dropped (17)

Numbers in brackets refer to `ai-errors.md`, F1–F5 to `review.md`.

Where did the AI save time? Generating the app from the one-page spec took 22 minutes: 16 Kotlin files, about 1,700 lines, and the first build succeeded without warnings (`generation-prompt.md`, `generation-report.md`). The AI drove the emulator over `adb` (permission dialogs, lifecycle, a removed sensor, injected values), so every check was repeatable and left log excerpts as evidence (`review.md`). One script it wrote ran all Part D experiments (5 runs per scenario, CPU time 3) unattended in about 20 minutes (CSV times 16:22–16:40). It also found and quoted sources quickly (Android docs and source code, GDPR).

Where did it make errors, and how were they noticed? `ai-errors.md` lists 18: 10 in tooling, 3 in the way of working, 3 in reasoning, 1 in the build and 1 in code; the review found two more in the code (F3, F4). Code and build errors were found by checks, not by reading the AI’s own report: the failing build, Android Lint and tests on the emulator. Tooling errors showed in the command output: timeouts, unreadable or cut-off screenshots, implausible results (100.3 m instead of 111.2 m in two GPS runs). Errors in the way of working and in reasoning were caught by the student, who challenged a weak argument, the way decisions were presented and work beyond the task; each of the three about the way of working became a rule in `CLAUDE.md`. The AI found one reasoning error itself by checking a hypothesis against the data (16). F1 and F2 were gaps in our own spec, not AI errors (no plausibility limit for GPS steps; a library list that contradicted the required APIs); both were fixed by spec amendments.